ROAMINGPIGS RESEARCH RECORD

Vulkan or ROCm? Speed and energy per token on one mini PC

Speed and energy per token for four local models on one AMD Ryzen AI MAX+ 395 mini PC, measured at the wall plug and at the chip, on two software backends, with the plan and the expectations written down before the run.

Self-published. Not peer reviewed. One machine.

18 W
at the wall when the machine sits idle

1.7 to 4.8
joules per token, net of idle, across all models and contexts

88 to 99%
ROCm decode speed as a share of Vulkan

73
tokens per second at best

48
measured cells on Vulkan, 48 on ROCm

7 of 8
expectations written in advance held

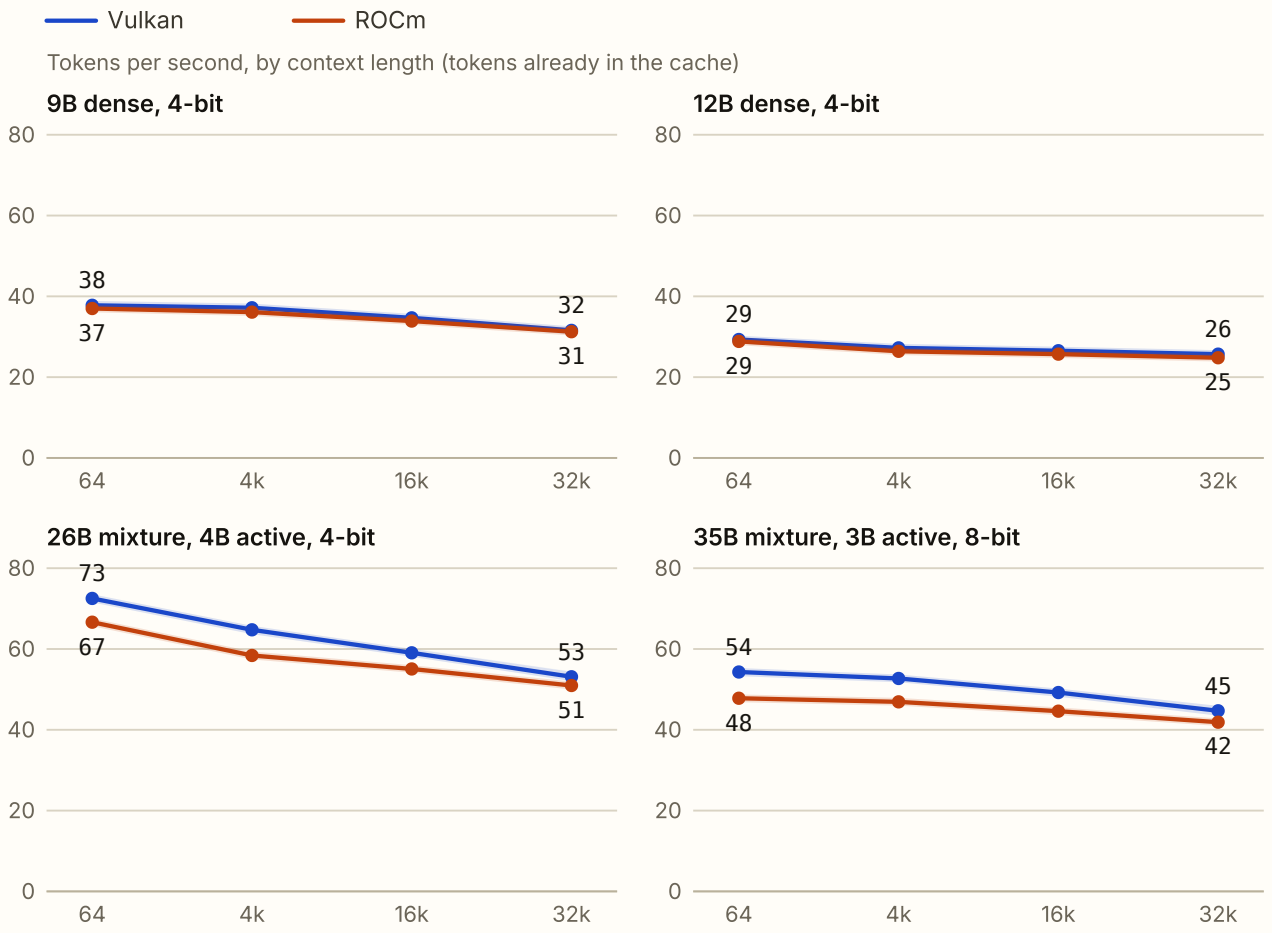
1.7%
gap between the plug's own energy counter and the sum of its readings

What I found

- **Speed.** ROCm decoded more slowly than Vulkan in 16 of 16 cells, by 1 to 12%, and the repeat ranges of the two backends never overlap in those cells. It was never faster.
- **Energy, 9B dense.** ROCm used more energy per token than Vulkan: +8 to +20% across the four context lengths (wall plug, net of idle).
- **Energy, 12B dense.** ROCm used about the same (the repeat ranges overlap in some cells) energy per token than Vulkan: -6 to +3% across the four context lengths (wall plug, net of idle).
- **Energy, 26B mixture.** ROCm used about the same (the repeat ranges overlap in some cells) energy per token than Vulkan: -6 to -1% across the four context lengths (wall plug, net of idle).
- **Energy, 35B mixture.** ROCm used less energy per token than Vulkan: -8 to -6% across the four context lengths (wall plug, net of idle).

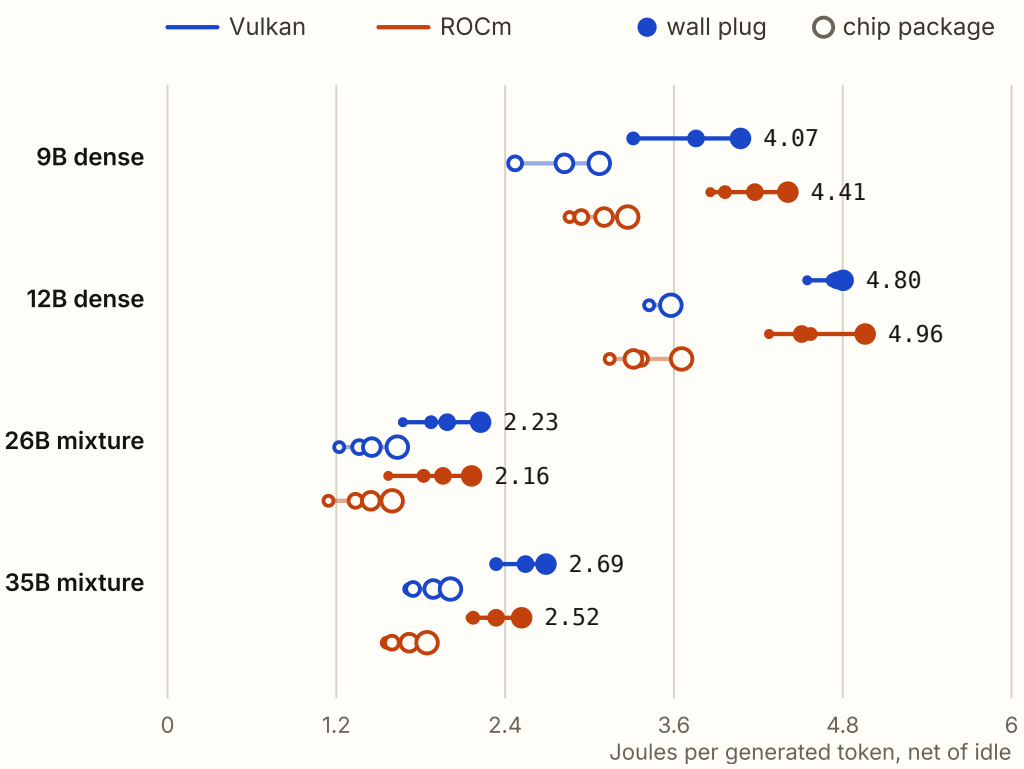
- **Repeatability.** Vulkan on the newer kernel, on another day, matched study 1 within 1.52% in every one of the 16 cells. Idle power was 17 W against 16.5 W.

Speed against context length



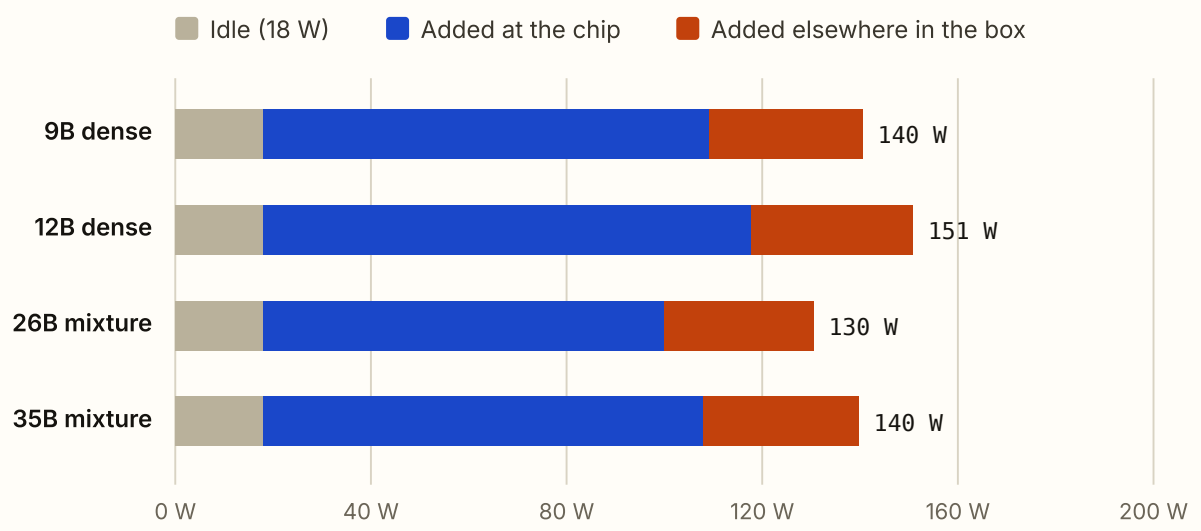
Decode speed. Tokens per second for each model as the prompt already in the cache grows from 64 to 32,768 tokens. The band is the lowest to highest of three repeats. All four panels share one scale.

Energy per token



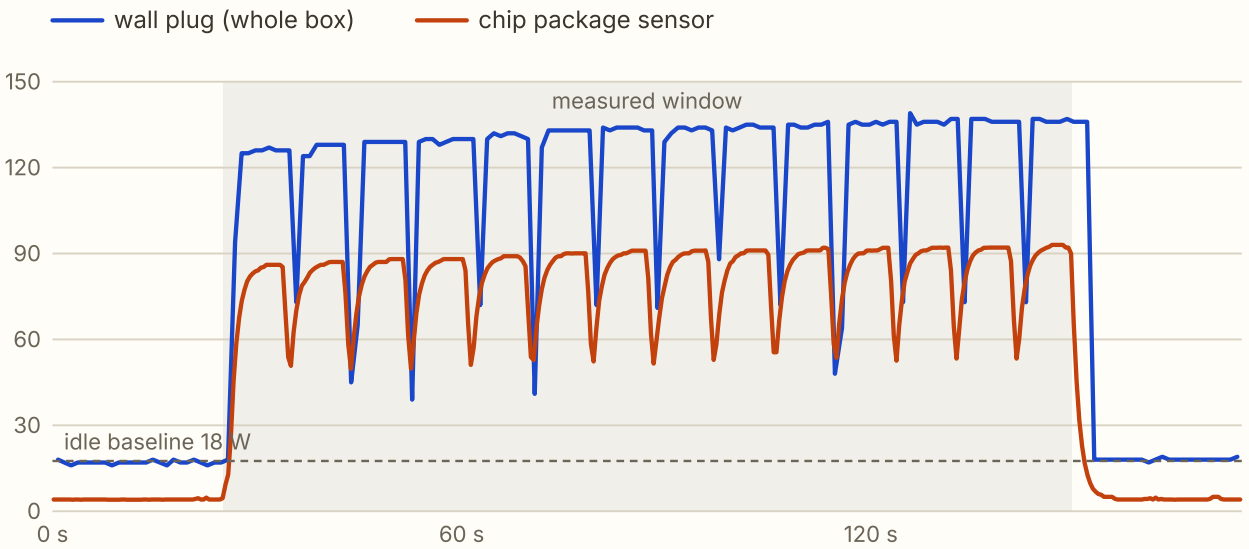
Joules per generated token, net of idle. Filled dots are the wall plug, hollow dots the chip's own sensor, each from 64 tokens of context (small dot) to 32k (large). The chip sensor always reads lower than the plug, because it sees only the chip.

Where the watts go



While decoding (Vulkan, 64 tokens of context). The whole box draws about 18 W idle. The chip package accounts for most of what is added. The rest is everything else in the box that the chip sensor does not see.

How one cell was measured



Watts against time. Net energy = area above the dashed line, inside the window, divided by tokens generated.

One measured cell, raw. The plug is read about once a second and the chip sensor ten times a second. Each dip is the gap between one 512-token request and the next. Energy per token is the area above the idle line inside the window, divided by the tokens generated in it.

ROCm against Vulkan

	64	4k	16k	32k
9B dense	98%	97%	98%	99%
12B dense	98%	97%	97%	96%
26B mixture	92%	90%	93%	96%
35B mixture	88%	89%	91%	94%

ROCm decode speed as a share of Vulkan. A plain number means the repeat ranges of the two backends do not overlap. A dotted outline means they overlap, so I make no claim.

Expectations written in advance

P1	Vulkan on kernel 38 within 3% of study 1 in every cell largest difference 1.52%	held
P2	ROCm within 25% of Vulkan; slower for the 26B mixture at 64 ROCm at 88 to 99% of Vulkan	held
P3	ROCm gap shrinks or reverses with context for some model	held
P4	Idle under 25 W and within 2 W of study 1, both backends idle 17 W Vulkan, 17 W ROCm, 16.5 W in study 1	held
P5	Wall power within 15% between backends, same cell ROCm wall power 87 to 117% of Vulkan	wrong
P6	Models rank the same by energy per token in both backends	held
P7	Repeat spread under 5% in every cell largest spread 1.26%	held
P8	All 96 cells, 3 repeats; parse error only with the 26B mixture 12 requests recovered after the parse error, 0 failed cells, 0 contaminated	held

Each line was written before the run. "Wrong" is reported, not hidden.

All numbers

Model	Context	tok/s	Wall W	Package W	Wall net J/token	Package net J/token	Repeats
9B dense, 4-bit	64	37.8	140	95	3.30	2.47	3
9B dense, 4-bit	4k	37.2	139	94	3.31	2.47	3
9B dense, 4-bit	16k	34.7	146	100	3.76	2.82	3
9B dense, 4-bit	32k	31.6	145	99	4.07	3.07	3
12B dense, 4-bit	64	29.3	151	104	4.55	3.42	3
12B dense, 4-bit	4k	27.3	147	101	4.73	3.58	3
12B dense, 4-bit	16k	26.5	145	99	4.76	3.58	3
12B dense, 4-bit	32k	25.7	141	96	4.80	3.58	3

26B mixture, 4B active, 4-bit	64	72.5	130	86	1.67	1.22	3
26B mixture, 4B active, 4-bit	4k	64.7	126	83	1.87	1.36	3
26B mixture, 4B active, 4-bit	16k	59.0	124	81	1.99	1.45	3
26B mixture, 4B active, 4-bit	32k	53.1	125	83	2.23	1.63	3
35B mixture, 3B active, 8-bit	64	54.3	140	94	2.33	1.72	3
35B mixture, 3B active, 8-bit	4k	52.7	138	93	2.34	1.75	3
35B mixture, 3B active, 8-bit	16k	49.2	140	94	2.54	1.89	3
35B mixture, 3B active, 8-bit	32k	44.7	135	91	2.69	2.01	3

ROCm

Model	Context	tok/s	Wall W	Package W	Wall net J/token	Package net J/token	Repeats
9B dense, 4-bit	64	37.0	161	109	3.86	2.86	3
9B dense, 4-bit	4k	36.1	163	110	3.96	2.94	3
9B dense, 4-bit	16k	33.9	161	109	4.18	3.10	3
9B dense, 4-bit	32k	31.3	157	106	4.41	3.27	3
12B dense, 4-bit	64	28.8	142	94	4.28	3.14	3
12B dense, 4-bit	4k	26.4	140	92	4.57	3.37	3
12B dense, 4-bit	16k	25.7	134	89	4.51	3.31	3
12B dense, 4-bit	32k	24.8	142	94	4.96	3.65	3
26B mixture, 4B active, 4-bit	64	66.6	123	80	1.57	1.14	3
26B mixture, 4B active, 4-bit	4k	58.4	125	82	1.82	1.34	3
26B mixture, 4B active, 4-bit	16k	55.1	127	83	1.96	1.44	3
26B mixture, 4B active, 4-bit	32k	51.0	129	85	2.16	1.60	3

35B mixture, 3B active, 8-bit	64	47.8	121	78	2.15	1.56	3
35B mixture, 3B active, 8-bit	4k	46.9	122	79	2.17	1.60	3
35B mixture, 3B active, 8-bit	16k	44.6	124	80	2.34	1.72	3
35B mixture, 3B active, 8-bit	32k	41.9	125	81	2.52	1.84	3

What this does not show

- Anything about other hardware, other chips, discrete GPUs or other builds of `llama.cpp`.
- The accuracy of the wall watts. The plug is uncalibrated and reads whole watts. The chip sensor is what the chip reports about itself.
- Batched or concurrent serving. Everything here is one stream at a time.
- Quality of the output. Prompts are licence texts and the replies are not scored.
- That anyone outside this project has repeated it.

Reproduce or check

Plan, code, raw wall and package power logs and every deviation from the plan: github.com/RoamingPigs/strix-halo-energy. Report built from commit `674caef`. Self-published by Cisco Caceres.